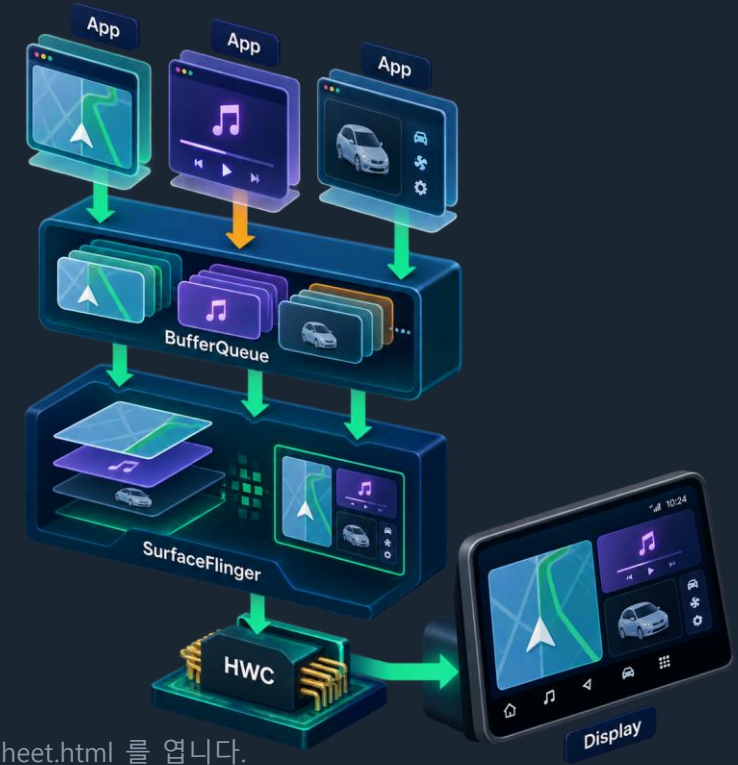


Android Framework 16 심화

Day 3 — sp/wp · Thread · Looper · JNI · SurfaceFlinger · 공유 메모리

libutils 3종 세트를 밑바닥부터 → Binder Thread 의 정체 → JNI → SurfaceFlinger Client (VSync + Looper) → pipe · ashmem · memfd → Memory / LMKD

애니메이션 · Worksheet 링크는 각 설명 슬라이드에 있으며, 이 파일과 같은 폴더의 Day3_Animation.html / Day3_Worksheet.html 를 엽니다.



오늘의 일정 — 8시간

Part	내용	소요
Part 0 환경·등록 실습	실습 1 환경 점검 · day3/Android.bp 일괄 등록 · 호스트 JDK	15분
Part 1 강의	Ch 1 libutils (RefBase·sp/wp·Thread·Looper) · Ch 2 JNI · Ch 3 Graphics · Ch 4 공유 메모리·Memory	115분
Part 2 실습	실습 2~12 (sp 4단계 → RefBase/wp → Thread → Looper → Binder 동작 → JNI → SurfaceFlinger → VSync → 공유 메모리 → 분석 /LMKD)	315분
Part 3 데모 · 정리	실습 13 Perfetto 합성 파이프라인·Jank · Worksheet	35분



오늘의 축: 실습 2~6 에서 만든 sp · Thread · Looper 가 실습 9~10 의 sp<SurfaceControl> · VSync Looper 로 그대로 재사용된다.

Part 0 · 실습 1 — 환경 점검 · Android.bp 일괄 등록 · JDK

1

빌드 환경 · Emulator 확인

lunch · \$OUT · getenforce · /dev/ashmem · memfd 심볼

2

호스트 JDK

sudo apt install default-jdk → javac -version (실습 7-8)

3

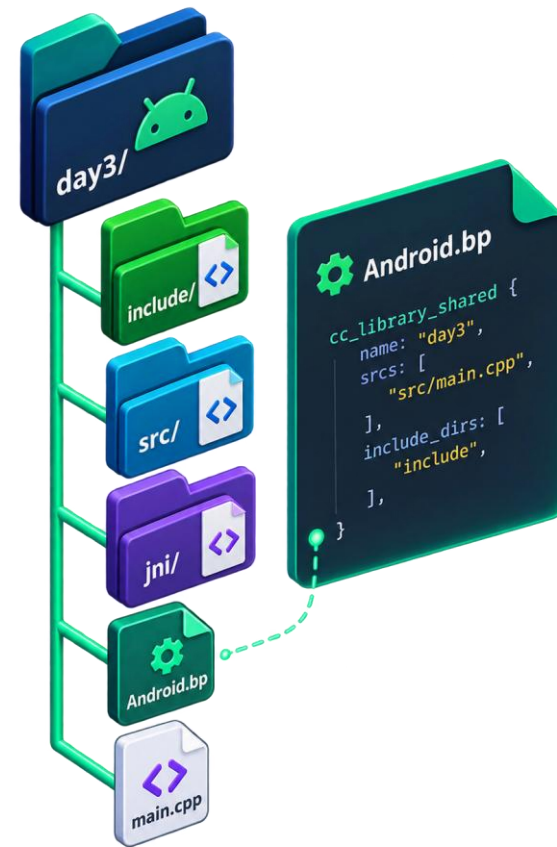
day3/ 디렉토리 + 빈 소스 + Android.bp 한 파일

sp_test · thread_test · loop_test · surface1 · vsync_anim · pipe · ashmem1/2 · memfd

4

Soong 분석 1회

m sp_test → analyzing Android.bp 통과 → 이후 .cpp 만 편집

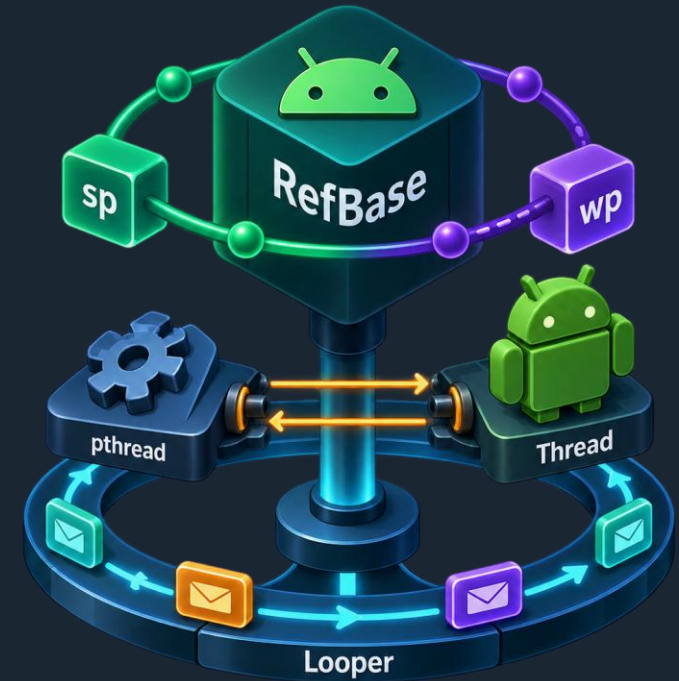


소요 15분 · 빌드는 소스 디렉토리에서 m <모듈명> — .bp 는 오늘 이 한 번만

Chapter 1

libutils — RefBase · sp/wp · Thread · Looper

40분 · 소유권(sp) · 실행 단위(Thread) · 이벤트 루프(Looper) — 프레임워크의 기본 골격 세 가지



sp<> 를 직접 만들어 보면 — RefBase 에 도달한다

① delete

sp 소멸자에서 delete
→ 복사하면 double free

② static 카운트

sp<T>::mRefs 공유
→ 객체 2개면 누수

③ 객체 안 카운트

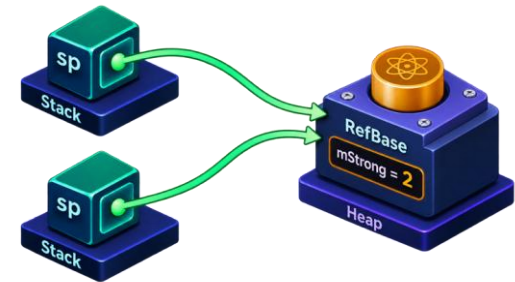
AAA 에 mRefs · incStrong
→ 클래스마다 반복

④ RefBase 분리

공통 부모 · delete this
= Android 구조

```
class RefBase {
    int mRefs;
    void incStrong() { mRefs++; }
    void decStrong() { if (--mRefs == 0) delete this; } // 객체가 스스로 사라진다
    virtual ~RefBase();
};

template<class T> class sp {
    sp(T* p) : mPtr(p) { mPtr->incStrong(); }    sp(const sp& r) { mPtr->incStrong(); }
    ~sp() { mPtr->decStrong(); }
};
```



- intrusive: 카운트가 객체 안에 → raw 를 다시 sp 로 감싸도 안전
- new 직후 sp 에 담고, delete 는 절대 쓰지 않는다
- Day 2 sp<IBinder> · 오늘 sp<SurfaceControl>·sp<Thread>·sp<Looper>

▶ 애니메이션 1 · sp 와 RefBase

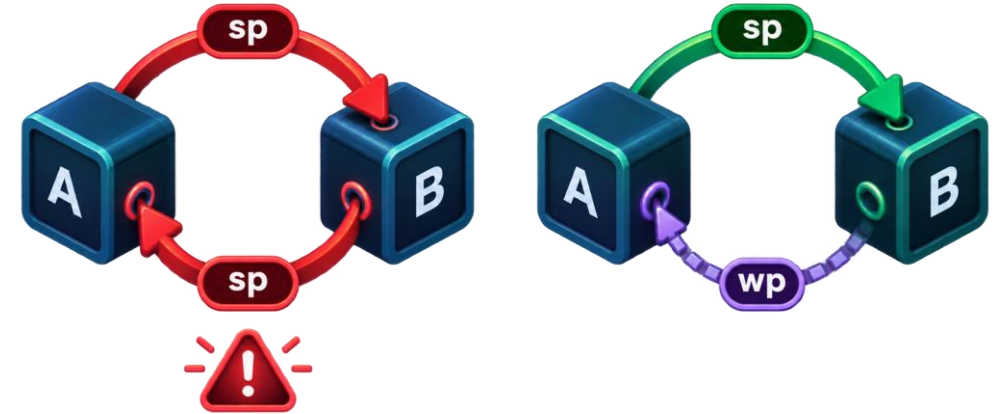
wp<> · promote() · 순환 참조

```

sp<AAA> owner = new AAA();           // mStrong 1
wp<AAA> p = owner;                   // 약한 참조 - 카운트 무관, -> 없음
sp<AAA> q = p.promote();              // 살아 있으면 sp (mStrong 2)
owner.clear(); q.clear();            // mStrong 0 → 소멸
sp<AAA> r = p.promote();              // ★ nullptr - 반드시 검사

// 순환 참조
class AAA : RefBase { sp<BBB> pb; };
class BBB : RefBase { sp<AAA> pa; }; // ★ 서로 잡음 → 영원히 안 죽음
class BBB : RefBase { wp<AAA> pa; }; // 한쪽을 wp 로 → 해결

```



- wp 는 객체가 죽은 뒤에도 안전하게 "죽었다" 를 안다 — mWeak 가 블록을 유지
- Listener · Callback · 부모↔자식 은 한쪽을 wp
- onFirstRef(): mStrong 0→1 순간 1회 — Thread::run 의 트리거 (다음 슬라이드)
- 실습 3: promote 실패 · 순환 누수 · wp 로 해결 3가지 확인

카운트	올리는 것	0 이 되면
mStrong	sp	객체 소멸 (delete this)
mWeak	sp + wp	weakref_impl (카운트 블록) 소멸

▶ 애니메이션 2 · wp · promote · 순환 참조

Thread 단계별 — pthread 에서 android::Thread 까지

① pthread

전역 foo → 멤버 접근 불가

② static + this

_handler(void*) 로 복원

③ virtual handler

골격은 부모, 내용은 자식

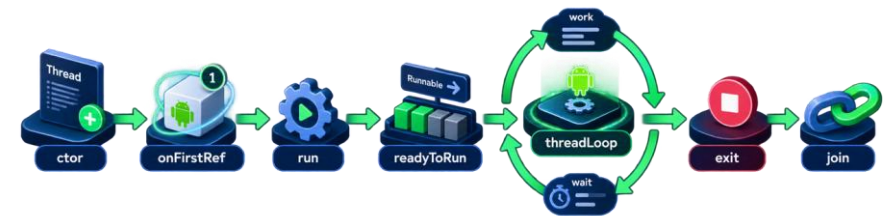
④ android::Thread

run · readyToRun · threadLoop

```

class MyThread : public Thread {
    void onFirstRef() override { run("my_thread"); } // sp 에 담기는
    순간 시작
    status_t readyToRun() override { return NO_ERROR; } // 1회 초기화
    bool threadLoop() override { // true → 반복,
        false → 종료
        sleep(1); return ++n < 3;
    }
};
sp<Thread> p = new MyThread; // ← 여기서 onFirstRef → run
p->join(); // threadLoop 이 false 를 반환할 때까지

```

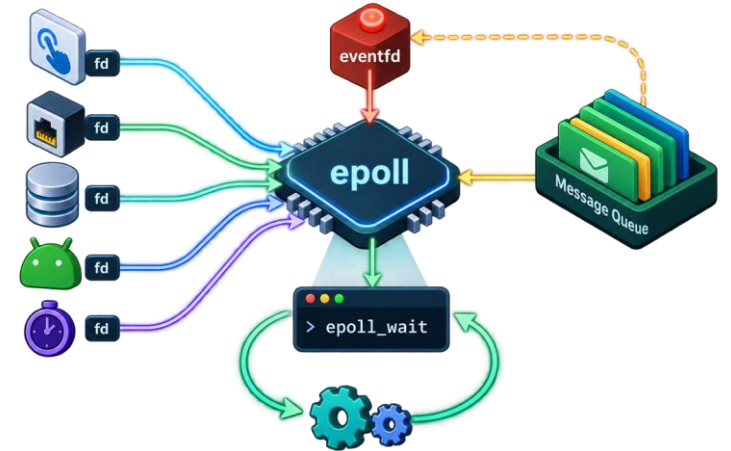


- static 트램폴린 + this: Looper 콜백·JNI·PoolThread 에 반복되는 관용구
- Thread : RefBase — run() 이 자기 자신을 sp 로 잡음(mHoldSelf) → main 이 sp 를 버려도 끝까지 돈다
- requestExit() 는 플래그만 — 루프에서 exitPending() 확인해 false 반환

▶ 애니메이션 3 · Thread 단계별

Looper 의 두 얼굴 — fd 이벤트 루프 + 메시지 큐

단계	Looper API	실체	pollOnce 반환
①	pollOnce(1000)	epoll_wait	POLL_TIMEOUT
②	다른 Thread 의 wake()	write(eventfd)	POLL_WAKE
③	addFd(fd, 0, EVENT_INPUT, foo, data)	epoll_ctl(ADD) → callback(fd, events, data)	POLL_CALLBACK
④	콜백을 클래스로 (static + this)	= LooperCallback::handleEvent	POLL_CALLBACK
⑤	sendMessage[Delayed](handler, Message)	큐에 넣고 wake → handleMessage(what)	(메시지 처리)



fd 이벤트 루프 (③④)

- VSync fd (실습 10)
- Input · Socket · pipe
- callback 은 Looper Thread 에서

메시지 큐 (⑤)

- uptime 순 정렬, 만기 시 handleMessage
- Java Handler.sendMessageDelayed 의 실체
- 한 Looper 에 두 기능

- Java Looper/Handler/MessageQueue = 이 C++ Looper 의 JNI wrapper (nativePollOnce)
- 실습 5 ⑤ 는 Message.what 으로 분기 — Day 1 Messenger 와 같은 모양

▶ 애니메이션 4 · Looper 의 두 얼굴

Binder Thread 와 Looper Thread — 두 개의 대기 루프

```
// ProcessState.cpp - Day 2 의 Binder Thread
class PoolThread : public Thread {                                // ← 실습 4 ④
    bool threadLoop() override {
        IPCThreadState::self()->joinThreadPool(mIsMain);    // Driver 대기 루프
        , 반환 안 함
        return false;
    }
};
// Java 콜백 → Main Thread
runOnUiThread(r) == new Handler(Looper.getMainLooper()).post(r)
    → enqueueMessage → nativeWake → C++ Looper::wake()    // ← 실습 5 ②
```



- Day 1 실습 9 의 runOnUiThread = Looper 큐에 넣기
- 실습 6: PoolThread · joinThreadPool · nativePollOnce 를 소스로
- startThreadPool 유무 → tid 분산/동일 (Day 2 서버 재사용)

	Binder Thread	Main (Looper) Thread
루프	joinThreadPool → ioctl 블로킹	Looper::pollOnce → epoll_wait
깨우는 것	다른 Process 의 transact	fd 이벤트 · wake · 메시지 만기
실행	BBinder::onTransact	handleEvent · handleMessage · UI
Looper	없다 → UI 접근 불가	있다

▶ 애니메이션 5 · Binder Thread vs Looper Thread

sp 4단계 직접 구현 → Android RefBase · wp

목표 자작 sp 로 RefBase 구조에 도달 (실습 2) → utils/RefBase.h 로 wp · promote · 순환 참조 (실습 3) · 소요 25 + 20분

진행 순서

1. 실습 2: sp_test/sp.cpp — #if 로 ① delete → ② static → ③ 객체 내 카운트 → ④ RefBase 전환
2. 실습 2: ① 복사 crash · ② 객체 2개 누수 · ④ ~RefBase 가 virtual 아니면? 실험
3. 실습 3: my_sp.cpp — Android sp / wp 만 → 즉시 소멸 / promote 성공·실패
4. 실습 3: A↔B sp 순환 → 소멸자 없음 → 한쪽 wp 로 해결
5. m sp_test my_sp_test → adb push → 실행

확인 포인트

- ② static: 소멸자 1번만
- ④: 소멸자 정확히 1번 + after
- ③ promote → nullptr, ④ 순환 누수, ⑤ 해결
- Day 2 sp<IBinder> 가 delete 없이 사라지는 이유 설명



▶ 애니메이션 1 · sp 와 RefBase

▶ 애니메이션 2 · wp · 순환 참조

Thread 단계별 → Looper 단계별

목표 pthread → android::Thread (실습 4) → pollOnce → wake → addFd → LooperCallback → MessageHandler (실습 5) · 소요 30 + 40분

진행 순서

1. 실습 4: thread.cpp — ① pthread → ② static __handler + this → ③ virtual handler → ④ android::Thread
2. 실습 4: my_thread.cpp — onFirstRef/readyToRun/threadLoop/join 로그 · sp 를 버려도 되는 실험(mHoldSelf)
3. 실습 5: Looper.cpp — ① pollOnce(1000) → ② wake → ③ addFd + pipe → ④ 콜백 클래스 → ⑤ sendMessage[Delayed]
4. 실습 5: strace -e epoll_wait,epoll_ctl,eventfd2 로 시스템 콜 확인
5. m thread_test my_thread_test looper_test → push → 실행

확인 포인트

- ④ sp<Thread> p = new MyThread; 만으로 시작
- readyToRun 1회 → threadLoop 3회 → join
- ① TIMEOUT ② WAKE ③ CALLBACK + data 0x1234
- ⑤ 1,2 즉시 · 3 은 3초 후



▶ 애니메이션 3 · Thread 단계별

▶ 애니메이션 4 · Looper 의 두 얼굴

Binder 에서의 동작 — PoolThread · joinThreadPool · Handler

목표 Day 2 Binder Thread 가 android::Thread 임을 소스로, Binder 콜백이 Looper 큐로 Main Thread 에 가는 경로 확인 · 소요 20분

진행 순서

1. :tag ProcessState::spawnPooledThread → PoolThread : Thread → threadLoop { joinThreadPool }
2. :tag IPCThreadState::joinThreadPool → talkWithDriver 루프 vs Looper::pollOnce 비교표
3. Emulator: my_server_test_4 → ps -T (binder:PID_N) · binder_logs thread 수 · 동시 3 client tid
4. startThreadPool 주석 처리 → 재빌드 → tid 동일·순차
5. :tag android_os_MessageQueue_nativePollOnce / NativeMessageQueue::wake

확인 포인트

- PoolThread 가 Thread 상속
- ps -T 에 binder:PID_N 이름 = Thread::run(name)
- startThreadPool 유무로 tid 비교
- nativePollOnce → Looper::pollOnce



▶ 애니메이션 5 · Binder Thread vs Looper

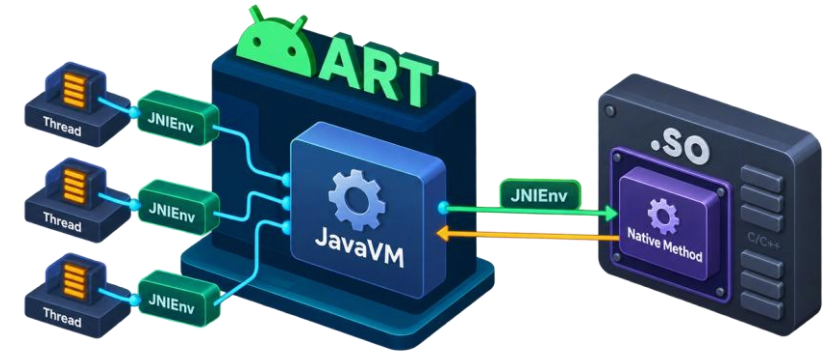
JNI — Java ↔ Native 경계

20분 · JavaVM / JNIEnv · 이름 규칙 vs RegisterNatives · 역방향 호출 · Invocation API · Framework JNI 세 지점



JNI 두 방향 — Native Method 와 역호출

```
// Java → C (First) 이름 규칙
JNIEXPORT void JNICALL Java_Hello_foo(JNIEnv* env, jobject thiz)
// 또는 JNI_OnLoad 에서 테이블 등록 (Third) - Framework 방식
JNINativeMethod nm[] = { {"foo", "()V", (void*)foo} };
env->RegisterNatives(clazz, nm, 1);
// C → Java (Second) 객체 생성
jclass cls = env->FindClass("JniTest");
jmethodID mid = env->GetMethodID(cls, "<init>", "(I)V");
jobject obj = env->NewObject(cls, mid, 100); // new JniTest(100)
```



- Invocation API: `JNI_CreateJavaVM` — Native 가 VM 을 만든다 → app_process 가 Zygote 를 띄우는 방식
- Framework JNI 세 지점: `JavaBinder(Binder)` · `NativeMessageQueue(Looper)` · `Surface.mNativeObject`
- "Java 객체 안의 long 하나 = C++ 객체 포인터"

개념	규칙
JavaVM / JNIEnv	Process 당 1 / Thread 당 1 (TLS) — JNIEnv 를 다른 Thread 에 넘기지 않는다
Local ref	jobject:jclass 는 함수 반환 시 무효 · 보관하려면 <code>NewGlobalRef</code>
시그니처	<code>(I)V</code> · <code>(Ljava/lang/String;)V</code> · <code>[I</code> — <code>javap -s</code> 로 확인
Native Thread 에서 Java	<code>AttachCurrentThread</code> + Global ref

▶ 애니메이션 6 · JNI 두 방향과 Framework JNI

JNI Native Method → Invocation API + Framework JNI

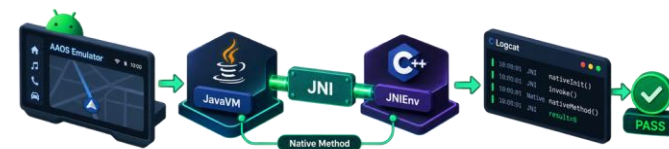
목표 First · Second · Third 세 단계 (실습 7) → JNI_CreateJavaVM · app_process · JavaBBinder 읽기 (실습 8) — 호스트 JDK · 소요 35 + 25 분

진행 순서

1. 실습 7 First: `javac -h` → `Java_Hello_foo` → `gcc -shared` → `LD_LIBRARY_PATH=. jav a Hello`
2. 실습 7 Second: `FindClass/GetMethodID("<init>")/NewObject` → Java 생성자 로그 · `javap -s`
3. 실습 7 Third: `JNI_OnLoad` + `RegisterNatives` — static 함수, `-h` 불필요
4. 실습 8: `invocation.cpp -ljvm` → Java main 호출 · `app_main.cpp` 의 `AndroidRuntime::start` 와 비교
5. 실습 8: `:tag JavaBBinder / android_os_MessageQueue_nativePollOnce / android_view_Surface_lockCanvas`

확인 포인트

- 헤더 함수명 = `Java_Hello_foo`
- C 가 만든 `JniTest(100)` 로그
- `RegisterNatives` 로 static foo 연결
- `JavaBBinder::onTransact` → `execTransact` 줄 찾기



▶ 애니메이션 6 · JNI

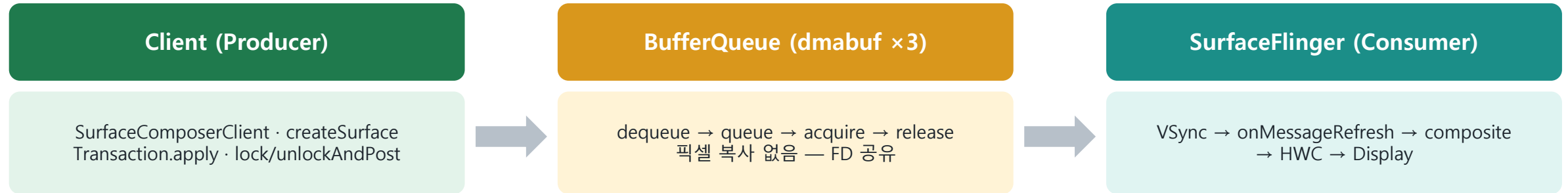
Chapter 3

Graphics — SurfaceFlinger · BufferQueue · VSync

25분 · Producer → BufferQueue → Consumer · Transaction · dmabuf · VSync fd 를 Looper 로



SurfaceFlinger — Producer → BufferQueue → Consumer



```

sp<SurfaceControl> sc = client->createSurface(String8("My Cpp Surface"), 400, 400, PIXEL_FORMAT_RGBA_8888, 0);
SurfaceComposerClient::Transaction tx;
tx.setLayer(sc, 0x7FFFFFFF).setPosition(sc, 100, 100).show(sc).apply(); // 한 VSync 에 원자 적용
sp<ANativeWindow> win = sc->getSurface();
ANativeWindow_lock(win.get(), &buf, nullptr); // dequeue
px = pixels + (y * buf.stride + x) * 4; // ★ stride ≠ width
ANativeWindow_unlockAndPost(win.get()); // queue → 다음 VSync 에 합성
  
```



- VSync fd = DisplayEventReceiver → Looper::addFd (실습 5 ④)
- Layer 수명 = sp<SurfaceControl> 수명
- 16.67 ms 예산 초과 → Jank (실습 13)

▶ 애니메이션 7 · SurfaceFlinger 파이프라인

SurfaceFlinger Client → VSync 애니메이션

목표 Layer 생성·Transaction·픽셀 그리기 (실습 9) → DisplayEventReceiver fd 를 LooperCallback 으로 매 프레임 이동 (실습 10) · 소요 30 + 30분

진행 순서

1. 실습 9: surface1/main.cpp — SurfaceComposerClient → createSurface(400×400) → Transaction → lock/unlockAndPost
2. 실습 9: 실험 — setLayer(0) · stride→width · while 삭제 · lock 두 번
3. 실습 10: vsync_anim.cpp — VSyncHandler : LooperCallback (실습 5 ④) · requestNextVsync · Looper::prepare + addFd + pollAll
4. 실습 10: setVsyncRate(2) · mX += 8 · usleep(20000) Jank · requestNextVsync 제거
5. m surface_client_test sf_vsync_anim → push → 실행

확인 포인트

- 빨간 400×400 사각형, 종료 시 사라짐
- dumsys SurfaceFlinger 에 Layer z-pos-stride
- 초록 사각형 10초 이동, --latency 16666667
- sp<SurfaceControl> vs unique_ptr<DisplayEventReceiver>



▶ 애니메이션 7 · SurfaceFlinger 파이프라인

▶ 애니메이션 4 · Looper (addFd)

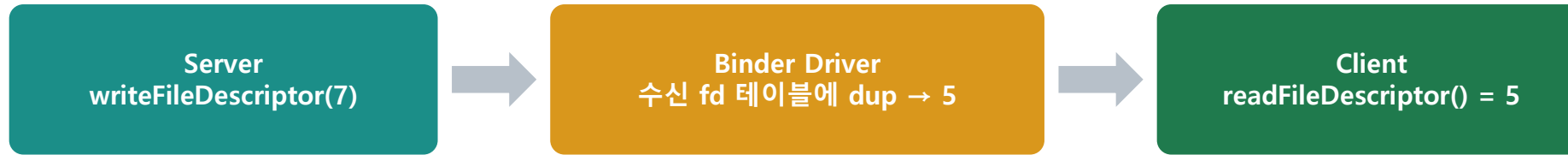
Chapter 4

공유 메모리 · Memory — pipe · ashmem · memfd · 측정 · LMKD

30분 · FD 가 Binder 를 타는 원리 · pipe 복사 vs ashmem/memfd 페이지 공유 · MemoryHeapBase · seal · PSS/USS · oom_score_adj



FD 하나가 Binder 를 탄다 — pipe · ashmem · memfd · dmabuf



숫자가 아니라 커널 file 객체에 대한 참조가 넘어간다 — 종류만 다르다

	pipe	ashmem / MemoryHeapBase	memfd + F_SEAL	dmabuf
데이터	복사 (링 버퍼)	없음 — 같은 페이지	없음	없음
접근	순차 · 1회	mmap 임의	mmap 임의	GPU-HW 도
보호	—	PROT_MASK 강등 · unpin 회수	F_SEAL_WRITE 커널 보장	—
Binder wrapper	ParcelFileDescriptor	IMemoryHeap / IMemory (interface_cast 가 mmap)	SharedMemory (Java)	GraphicBuffer
사용처	로그 · 스트림	AudioFlinger · Camera · CursorWindow	신뢰 경계 버퍼	BufferQueue · Codec



```

sp<IMemoryHeap> heap = new MemoryHeapBase(4096); // ashmem/memfd + mmap
addService("ashmem.service", IMemoryHeap::asBinder(heap));
// client
sp<IMemoryHeap> h = interface_cast<IMemoryHeap>(binder); // FD 수신 + mmap
char* p = (char*)h->getBase(); // 서버와 같은 페이지
  
```

▶ 애니메이션 8 · 공유 메모리 — FD 전달

Memory 측정 — PSS / USS · Zygote COW · oom_score_adj · LMKD

지표	정의	언제
VSS	가상 주소 공간	거의 안 봄
RSS	물리 페이지 (공유 중복 계산)	대략
PSS	공유 페이지를 n 분의 1 로 배분	프로세스 비교·합산
USS	이 프로세스만의 페이지	죽이면 돌아오는 양

LMKD

- PSI /proc/pressure/memory
- oom_score_adj 높은 순
- Cached 먼저, Foreground 마지막
- ro.lmk.* 로 조정

```
$ dumphys meminfo com.android.car
Native Heap · Dalvik Heap · .so mmap (shared clean ← Zygote COW)
Ashmem / Other mmap ← 실습 11 · Graphics ← 실습 9·10 · TOTAL PSS
$ procrank | head          # VSS RSS PSS USS
$ showmap <pid>           # 매핑별 · shared clean vs private dirty
$ cat /proc/<pid>/oom_score_adj # 0 Foreground ... 900+ Cached
$ logcat -s lmkd          # Kill 'com...settings' oom_score_adj 985 to free ...
```



- Zygote COW: preload 페이지가 shared clean → App PSS 가 작다
- 공유 메모리는 Native Heap 이 아닌 별도 항목
- AMS 가 Process 상태별로 oom_score_adj 부여 (dumphys s activity oom)
- 실습 12: MemHog 로 압박 → lmkd Kill 순서 관찰

공유 메모리 3단계 — pipe → ashmem → memfd

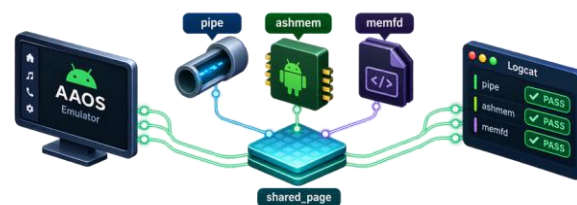
목표 Day 1 실습 11 의 "FD 만 Binder 로" 를 Native 에서 세 방식으로, /proc/<pid>/fd 와 dup 번호 확인 · 소요 30분

진행 순서

1. ① pipe_server/client: BBinder 가 pipe(fd) → writeFileDescriptor(fd[0]) · Writer : Thread(실습 4) 가 8 MB write · Client fd 번호 ≠ 서버 (dup)
2. ② ashmem1: MemoryHeapBase(4096) → addService(IMemoryHeap::asBinder) · Client interface_cast → getBase() 같은 페이지
3. ② ashmem2: MemoryBase(heap, 2048, 1024) → IMemory · getMemory(&offset, &size) · ls -l /proc/<pid>/fd
4. ③ memfd: memfd_create + ftruncate + mmap + F_ADD_SEALS → Client PROT_WRITE mmap / ftruncate → EPERM
5. m pipe_server pipe_client ashmem1_* ashmem2_* memfd_* → push → 각 서버 & 후 client

확인 포인트

- ① fd 번호 차이, 8388608 bytes 수신
- ② 서버 문자열을 Client 가 읽음
- ② offset=2048 size=1024
- ③ seals=0xf, EPERM 두 번
- 비교표 작성



▶ 애니메이션 8 · 공유 메모리 — FD 전달

Graphics · Memory 분석 + LMKD

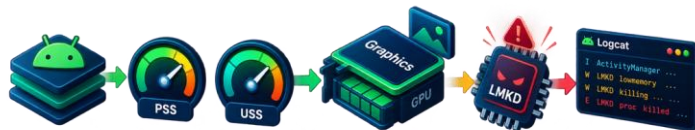
목표 `dumpsys SurfaceFlinger` 로 내 Layer, `meminfo/procrank/showmap` 으로 PSS·USS, `oom_score_adj` 와 `lmkd Kill` 순서 · 소요 30분

진행 순서

1. `surface_client_test` 띄운 채 `dumpsys SurfaceFlinger --list · grep -A25 'My Cpp Surface' (z · pos · stride 448) · --latency · gfxinfo`
2. `dumpsys meminfo com.android.car` 항목 해석 · `Ashmem/Other mmap` 에 실습 11 · Graphics 에 실습 9
3. `procrank · showmap <pid>` — Zygote 와 App 의 shared clean 비교
4. `oom_score_adj`: Foreground 0 → 홈으로 나가면 900 대 · `dumpsys activity oom`
5. MemHog (byte[50MB] 누적) → `logcat -s lmkd Kill` 순서 · `/proc/pressure/memory · getprop ro.lmk`

확인 포인트

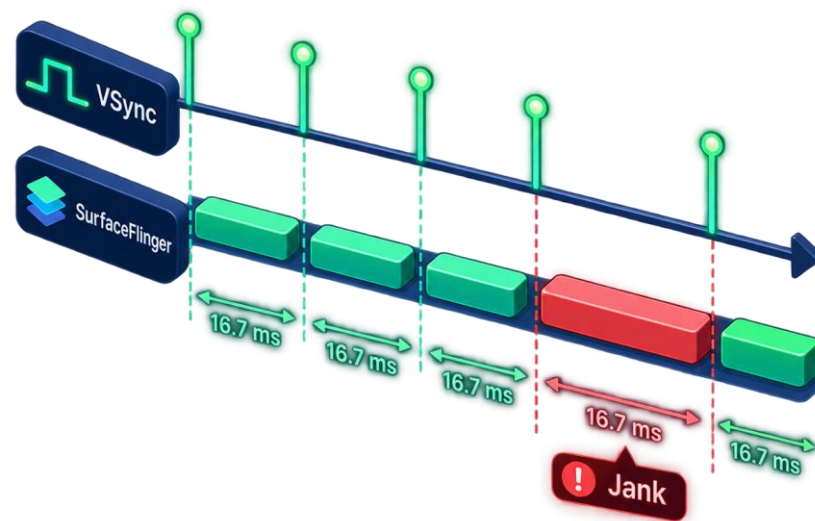
- Layer 의 activeBuffer stride 448 (≠400)
- `.so mmap` = shared clean
- `oom_score_adj` 0 ↔ 9xx 변화
- `lmkd Kill '...'` `oom_score_adj 9xx` 순서



실습 13 (데모) · Perfetto 로 합성 파이프라인 · Jank 보기

```
$ adb shell perfetto -c - --txt -o /data/misc/perfetto-traces/day3.perfetto-t
race <<EOF
buffers: { size_kb: 65536 fill_policy: RING_BUFFER }
data_sources: { config { name: "linux.ftrace" ftrace_config {
  atrace_categories: "gfx" atrace_categories: "sf" atrace_apps: "*" } } }
data_sources: { config { name: "android.surfaceflinger.frametimeline" } }
duration_ms: 6000
EOF
```

트랙	보는 것
vsync_rect Actual/Expected Timeline	프레임마다 예산 준수 여부 — 빨강 = Jank
surfaceflinger	onMessageRefresh · composite · HWC — 매 VSync 반복
sf_vsync_anim main	handleEvent 가 VSync 직후 짧게 → Transaction.apply
usleep(20000) 후	handleEvent 가 16.67 ms 초과 → 다음 VSync 침범 → 드롭



Day 3 Worksheet — 20문제, 20분

유형	문제	범위
객관식	Q1 ~ Q8	intrusive sp · promote · threadLoop 반환 · epoll · Binder Thread 와 Looper · RegisterNatives · dmabuf · F_SEAL
단답형	Q9 ~ Q13	onFirstRef · wake · PoolThread · JNI_CreateJavaVM · MemoryHeapBase
빈칸 채우기	Q14 ~ Q17	static/this · addFd/sendMessage · JavaBBinder/mNativeObject · Transaction/16.67
순번 지정	Q18 ~ Q19	Thread 생애주기 · 픽셀 → 화면
선잇기	Q20	pipe · ashmem · memfd · dmabuf · writeFileDescriptor ↔ 특징

- Day3_Worksheet.html 을 열어 풀고 채점하기 → 문제별 해설 확인
- 종이로 풀 경우 인쇄 (문제만) → 강사가 정답·해설 모두 보기로 풀이
- 16 / 20 이상이면 오늘 내용을 충분히 이해한 것



▶ [Day 3 Worksheet 열기](#)

오늘 확인한 것

sp / wp

카운트는 객체 안(RefBase, intrusive) · wp 는 promote 후 검사 · 순환은 한쪽 wp · onFirstRef

Thread · Looper

static+this 트램폴린 · run/readyToRun/threadLoop · Looper = epoll fd 루프 + 메시지 큐

Binder 와의 관계

Binder Thread = PoolThread(joinThreadPool) · Main = Looper::pollOnce · 콜백은 Handler.post 로 큐에

JNI · Graphics

Java_규칙 vs RegisterNatives · long → C++ 객체 · Producer→BufferQueue→SF · VSync fd 를 addFd

공유 메모리

FD 참조가 dup 되어 넘어간다 · pipe 복사 / ashmem·memfd 페이지 공유 / seal / dmabuf

